

Prompt para IA de Desenvolvimento – Otimização Avançada do Buffer e Inicialização da Reprodução das Rádios

Objetivo

Realizar uma auditoria completa no sistema de reprodução de áudio do aplicativo para identificar e corrigir a causa da demora no início da reprodução de algumas emissoras.

Nos testes realizados, os mesmos links de streaming iniciam a reprodução quase instantaneamente em outros aplicativos e players (como VLC, Winamp, AIMP, Foobar2000, ExoPlayer, VLC Mobile, TuneIn, Radio Garden e players HTML5 otimizados), demonstrando que o problema **não está nos servidores das rádios nem no buffer natural dos streams**, mas sim na forma como o aplicativo gerencia a reprodução e o buffer.

O objetivo é reduzir significativamente o tempo necessário para iniciar a reprodução, mantendo a estabilidade e a qualidade do áudio.

Diagnóstico Inicial

Antes de implementar qualquer alteração, faça uma análise completa do pipeline de reprodução para identificar gargalos relacionados a:

- inicialização do player;
- criação das conexões HTTP/HTTPS;
- tempo de resolução DNS;
- tempo de handshake TLS;
- carregamento do elemento de áudio;
- criação do MediaSource;
- configuração do buffer;
- tempo até o primeiro pacote de áudio (First Audio Frame);
- estratégias de pré-carregamento;
- reconexões desnecessárias;
- latência introduzida pelo código da aplicação.

Gerar um relatório interno indicando onde ocorre o maior atraso.

Auditoria do Player

Verifique se existem configurações que aumentam desnecessariamente o tempo de espera, como:

- buffer inicial muito grande;

- pré-carregamento excessivo;
- tempo de espera antes do `play()`;
- retries desnecessários antes da primeira reprodução;
- bloqueios causados por promises;
- filas de eventos;
- múltiplas inicializações do player;
- reconfiguração do player a cada troca de rádio;
- uso inadequado de `preload`;
- listeners duplicados;
- reconexões desnecessárias.

Corrija automaticamente qualquer configuração que aumente a latência.

Redução Inteligente do Buffer

O aplicativo deve iniciar a reprodução com o menor buffer possível, sem comprometer a estabilidade.

Priorizar:

- menor tempo até o primeiro áudio;
- carregamento progressivo;
- preenchimento do buffer durante a reprodução.

Evitar aguardar grandes quantidades de dados antes de iniciar o áudio.

Implementação de Bibliotecas Especializadas

Sempre que compatível com o tipo de stream, implemente suporte às melhores bibliotecas para reprodução de mídia.

Avalie automaticamente qual utilizar em cada situação.

Priorize:

HLS

Implementar suporte ao **Hls.js** para streams HLS (.m3u8).

Configurar o Hls.js para baixa latência (Low-Latency HLS) sempre que suportado pelo stream.

MPEG-DASH

Implementar suporte ao **Dash.js** para streams MPEG-DASH (.mpd).

Utilizar configurações otimizadas para início rápido da reprodução.

Video.js

Caso o projeto utilize um player HTML5 customizado, integrar **Video.js** quando ele proporcionar melhor compatibilidade e desempenho, utilizando-o apenas como camada de reprodução e mantendo a interface existente.

Shaka Player

Adicionar suporte ao **Shaka Player** para conteúdos DASH e HLS quando oferecer melhor desempenho ou recuperação automática de erros.

HTML5 Audio

Para streams MP3, AAC, OGG e ICY que não necessitem de bibliotecas específicas, manter o elemento nativo `<audio>` sempre que ele for mais rápido que bibliotecas externas.

Não utilizar uma biblioteca apenas por utilizá-la; escolha automaticamente a solução mais eficiente para cada protocolo.

Seleção Inteligente do Player

Criar um mecanismo de seleção automática.

Exemplo:

- `.m3u8` → Hls.js
- `.mpd` → Dash.js ou Shaka
- MP3/AAC/ICY → HTML5 Audio otimizado

Caso uma biblioteca não seja compatível com determinado stream, utilizar automaticamente a melhor alternativa disponível.

Configuração de Baixa Latência

Sempre que possível, configurar:

- buffer inicial reduzido;
- carregamento progressivo;
- baixa latência;
- início imediato da reprodução;
- recuperação automática de pequenas interrupções.

Evitar configurações voltadas para vídeos longos ou VOD.

Pré-Carregamento Inteligente

Quando uma rádio estiver selecionada:

- iniciar discretamente a preparação da conexão;
- resolver DNS antecipadamente;
- preparar a conexão HTTP/HTTPS;
- reutilizar conexões persistentes quando possível.

Não iniciar o download do stream antes da seleção do usuário, mas minimizar o tempo entre o clique e o início da reprodução.

Reutilização do Player

Evitar destruir e recriar completamente o player a cada troca de rádio.

Sempre que possível:

- reutilizar a mesma instância;
 - trocar apenas a fonte (`src`);
 - limpar corretamente os buffers anteriores;
 - preservar a configuração do player.
-

Cache Inteligente

Implementar cache apenas para recursos auxiliares, como:

- DNS;
- conexões persistentes;
- certificados TLS;
- informações das emissoras.

Nunca armazenar em cache o áudio ao vivo.

Timeout

Reduzir tempos de espera excessivos antes de considerar uma tentativa malsucedida.

Evitar bloqueios longos durante a conexão inicial.

Recuperação Automática

Caso o stream apresente falha:

- reconectar rapidamente;
 - reutilizar conexões quando possível;
 - evitar reinicializar completamente o player.
-

Compatibilidade

Garantir suporte para:

- MP3
 - AAC
 - AAC+
 - OGG
 - Opus
 - ICY
 - HLS
 - MPEG-DASH
 - HTTPS
 - HTTP
-

Performance

A implementação deve:

- reduzir consumo de CPU;
 - reduzir consumo de memória;
 - evitar criação de múltiplos players;
 - evitar múltiplas conexões simultâneas;
 - eliminar vazamentos de memória.
-

Logs de Diagnóstico

Adicionar logs internos (somente para depuração) registrando:

- tempo entre o clique e o início da reprodução;
- tempo de conexão;
- tempo de carregamento do buffer;
- tempo até o primeiro pacote de áudio;
- tempo até o primeiro som reproduzido;
- player utilizado (HTML5, Hls.js, Dash.js, Shaka ou outro);
- motivo de eventuais atrasos.

Esses logs não devem ser exibidos ao usuário.

Testes Comparativos

Após a implementação, realizar testes comparativos entre:

- versão anterior do aplicativo;
- nova versão otimizada.

Medir indicadores como:

- tempo médio até o início da reprodução;
- tempo de criação da conexão;
- tempo até o primeiro áudio;
- estabilidade da reprodução;
- consumo de memória;
- consumo de CPU.

Caso alguma biblioteca aumente a latência para determinado protocolo, ela não deverá ser utilizada nesse cenário.

Preservar

Não alterar:

- interface do aplicativo;
- player visual;
- favoritos;
- histórico;
- categorias;
- listas de rádios;
- APIs;
- lógica de autenticação;
- identidade visual.

Modificar apenas o mecanismo interno de reprodução e gerenciamento do buffer.

Validação Final

Ao concluir a implementação, confirme que:

- o tempo de início da reprodução foi significativamente reduzido;
- o buffer inicial foi otimizado sem comprometer a estabilidade;
- o player seleciona automaticamente a tecnologia mais adequada para cada tipo de stream;
- Hls.js, Dash.js, Shaka Player, Video.js (quando aplicável) e o player HTML5 são utilizados de forma inteligente e apenas quando oferecem vantagem real;
- não existem regressões de desempenho;

- a reprodução inicia de forma comparável aos principais aplicativos de rádio disponíveis no mercado.

Objetivo Final

Entregar um sistema de reprodução altamente otimizado, com inicialização rápida, gerenciamento inteligente de buffer e seleção automática da melhor tecnologia de reprodução para cada protocolo de streaming, eliminando atrasos introduzidos pela aplicação e proporcionando uma experiência equivalente à dos melhores players de rádio do mercado, sem alterar a interface ou a lógica de negócio do aplicativo.